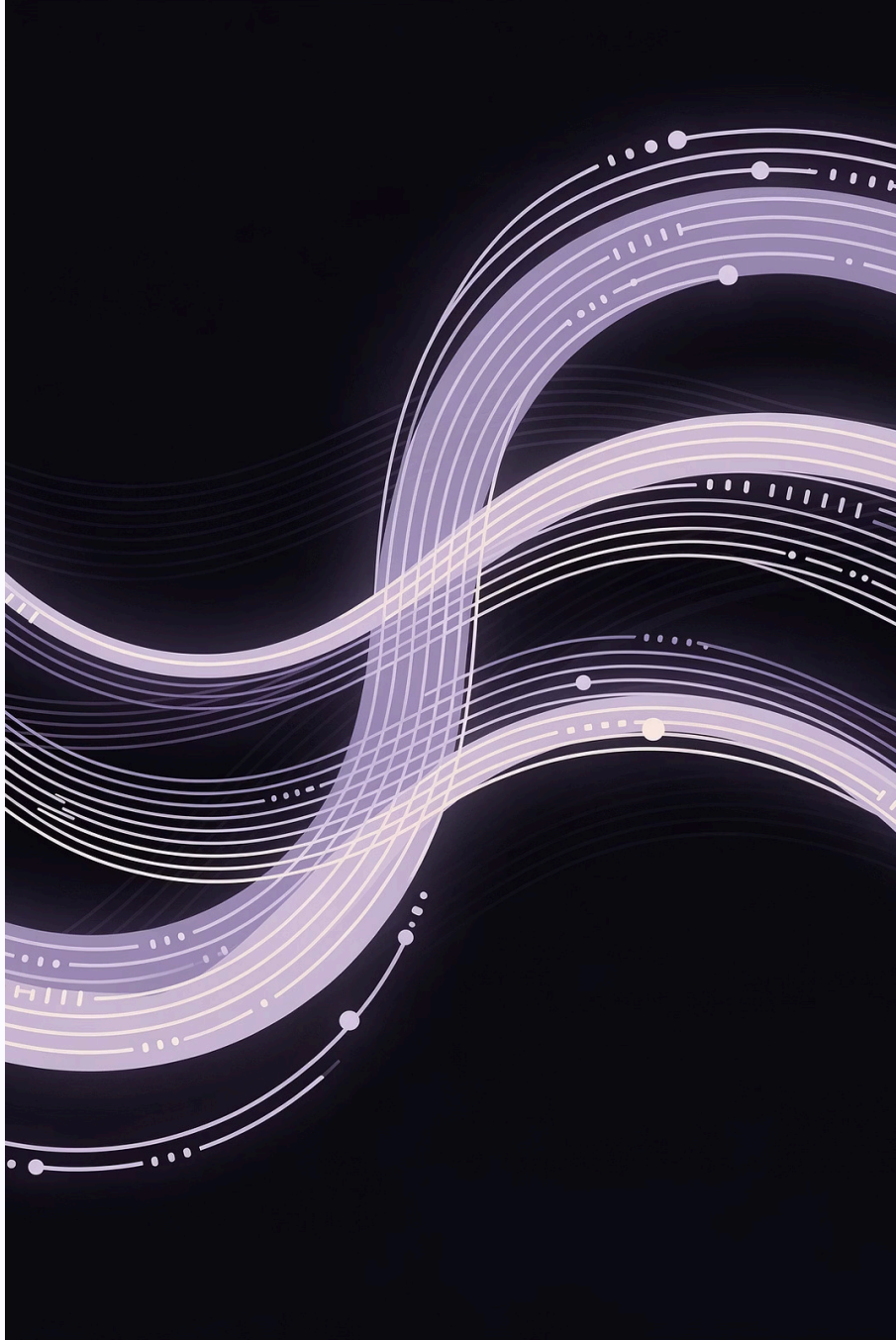
Real-Time Data Processing

From batch pipelines to continuous event-driven architectures — understanding how modern data systems process information the moment it arrives.

STREAMING FUNDAMENTALS



CONTEXT

Building on What We Know

We've explored how data can be moved and transformed in structured batches. Now we push further — into a world where data never stops arriving.

Batch Pipelines

Processing data in scheduled, fixed-size chunks — reliable, but inherently delayed.

DLT Pipelines

Delta Live Tables for declarative, managed ETL with lineage and quality constraints.

Streaming

What if data arrives continuously? We process it the moment it lands.

Batch vs. Streaming

These two paradigms represent fundamentally different assumptions about *when* data is available and *how quickly* decisions need to be made.

⏱️ **Batch Processing**	⚡ **Stream Processing**
Data collected over a period, then processed in one run	Data processed continuously as each event arrives
High latency — minutes to hours between data and insight	Low latency — milliseconds to seconds end-to-end
Simpler to reason about; easier to reprocess and debug	More complex; requires windowing, state, and fault tolerance
Ideal for end-of-day reports, ETL jobs, model training	Ideal for fraud detection, live dashboards, alerting
Resources used in bursts on a schedule	Resources consumed continuously around the clock

The Limitations of Batch Processing

Batch pipelines are powerful and widely used — but when the business demands *immediate* action on fresh data, their fundamental design becomes a bottleneck.

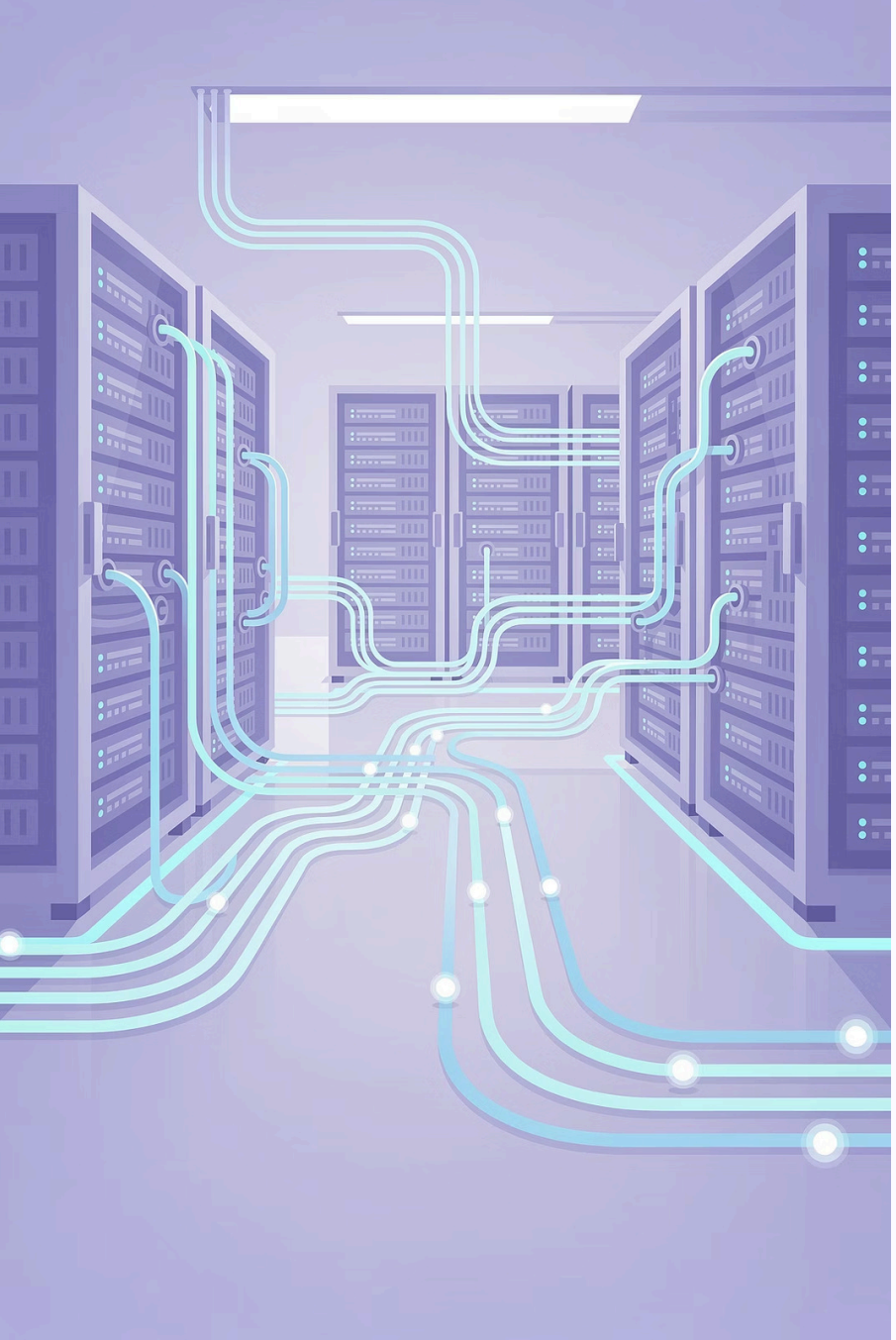
The Latency Problem

Batch jobs run on a schedule — hourly, nightly, or weekly. Every second between an event occurring and the pipeline running is a second of blind spot. In fast-moving domains, that delay is unacceptable.

Delayed Insights

By the time a batch pipeline surfaces an anomaly — a fraudulent transaction, a failing sensor, a surge in traffic — the window to act has often already closed. Decisions made on stale data are decisions made in the past.

> Batch pipelines answer: **"What happened?"** Streaming answers: **"What is happening right now?"**



What Is Stream Processing?

Continuous Data Processing

Rather than accumulating records and processing them in bulk, stream processing handles each event — or a micro-batch of events — as it arrives. The pipeline is always running, always consuming, always producing results.

Event-Driven by Design

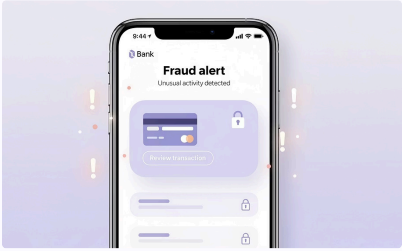
Every message, transaction, click, or sensor reading is an **event**. Stream processing systems are built around reacting to these events in near real-time, enabling downstream systems to act without waiting for a scheduled job to kick off.

Stateful and Windowed

Streaming systems can maintain state across events and group records into time windows (tumbling, sliding, session) — enabling aggregations like "count of transactions in the last 60 seconds" without storing the entire dataset.

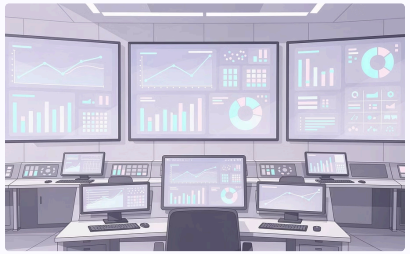
Real-World Streaming Use Cases

Stream processing is the backbone of many systems where acting on data *immediately* creates tangible business value or prevents serious harm.



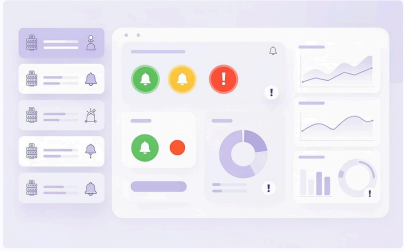
Fraud Detection

Flag suspicious transactions the moment they occur — before they clear. Rule-based and ML models evaluate each event in milliseconds, triggering holds or alerts in real time.



Real-Time Analytics

Power live dashboards with up-to-the-second metrics — active users, revenue per minute, conversion rates — without waiting for batch aggregations to complete.

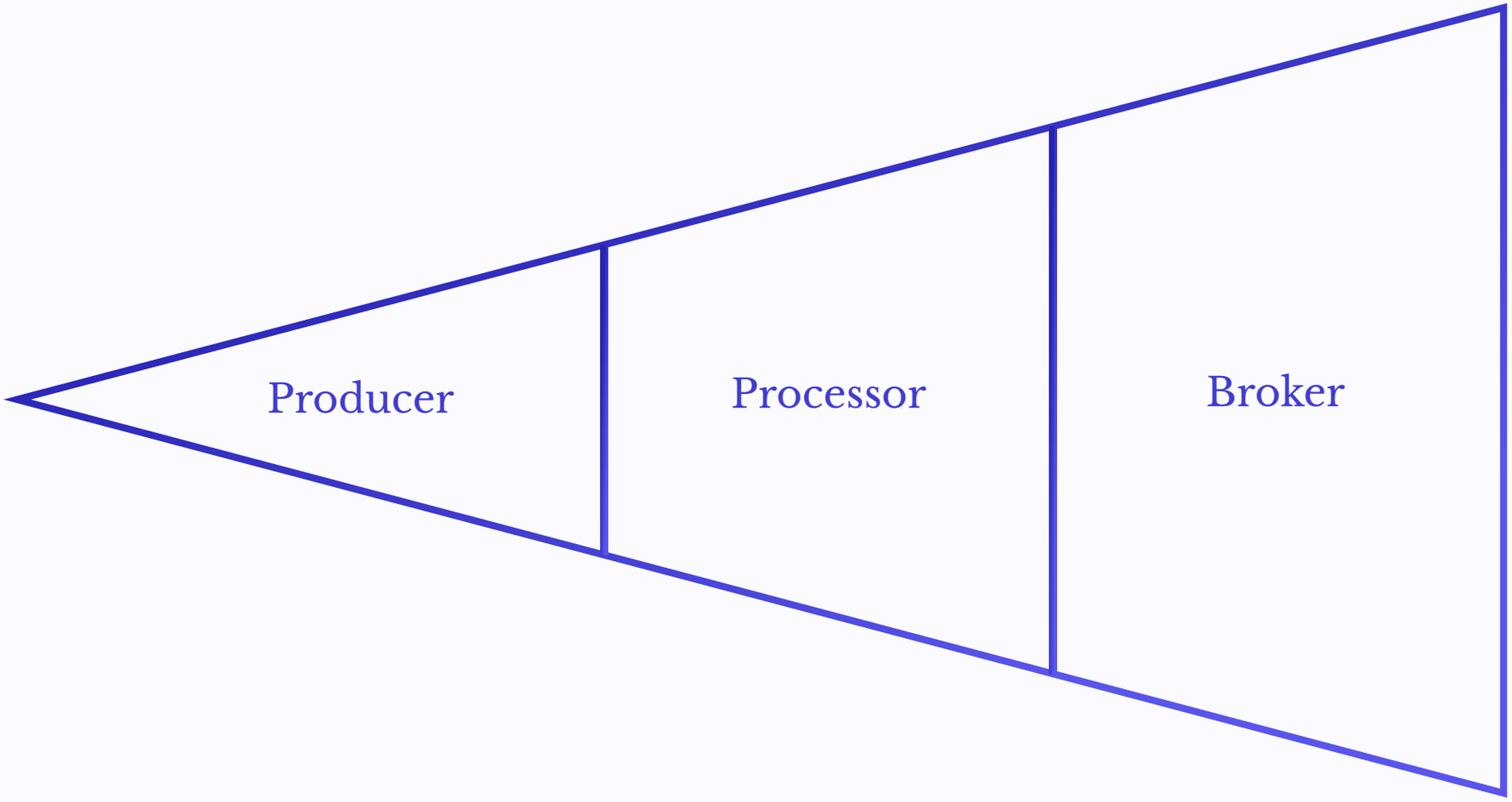


Infrastructure Monitoring

Ingest logs and metrics from thousands of services continuously. Detect latency spikes, error surges, or resource exhaustion and trigger automated remediation before users are impacted.

Streaming Architecture at a Glance

A streaming pipeline follows a consistent pattern: data is produced, brokered reliably, processed with logic, and persisted for downstream consumption.



Each layer has a distinct responsibility. The **broker** (e.g. Apache Kafka) decouples producers from consumers, ensuring durability and replay. The **processor** (e.g. Spark Structured Streaming) applies business logic. **Storage** makes results queryable by downstream systems.

Micro-Batch vs. Continuous Processing

Not all streaming engines work the same way under the hood. Understanding *how* your engine processes data affects latency guarantees, throughput, and operational complexity.

Spark: Micro-Batch Model

Spark Structured Streaming collects incoming events into small, discrete time intervals — called **micro-batches** — and processes each one as a mini Spark job. This delivers near-real-time latency (typically seconds) while reusing Spark's proven batch execution engine, fault tolerance, and exactly-once semantics.

True Continuous Processing

Engines like Apache Flink operate in a fully continuous model — processing each record the instant it arrives, with no batching interval. This achieves sub-millisecond latency but introduces more complexity in state management and recovery. Spark introduced an experimental continuous mode, though micro-batch remains the production-recommended approach.

Key trade-off: Micro-batch trades a small amount of latency for significantly simpler operations and stronger consistency guarantees — the right choice for most production workloads.

Core Challenges in Stream Processing

Building a streaming pipeline that is correct, reliable, and efficient in production requires confronting problems that simply don't exist in the batch world.

Late-Arriving Data

Events may arrive out of order due to network delays or upstream failures. Streaming systems must define a **watermark** — a threshold of acceptable lateness — and decide how to handle records that arrive after a window has closed.

Fault Tolerance

A streaming job runs indefinitely. Worker nodes can crash, network partitions occur, and upstream topics can lag. Systems must be designed to **checkpoint state** and recover from failures without losing or duplicating records.

Exactly-Once Processing

Guaranteeing that each event is processed *exactly once* — not zero times (data loss) or more than once (duplicates) — is one of the hardest problems in distributed systems. Spark + Kafka together provide exactly-once end-to-end semantics when configured correctly.

Time to Build

Let's implement streaming with Spark + Kafka

We've covered the theory — batch limitations, streaming fundamentals, architecture patterns, and the challenges you'll face in production. Now it's time to put it into practice.

01

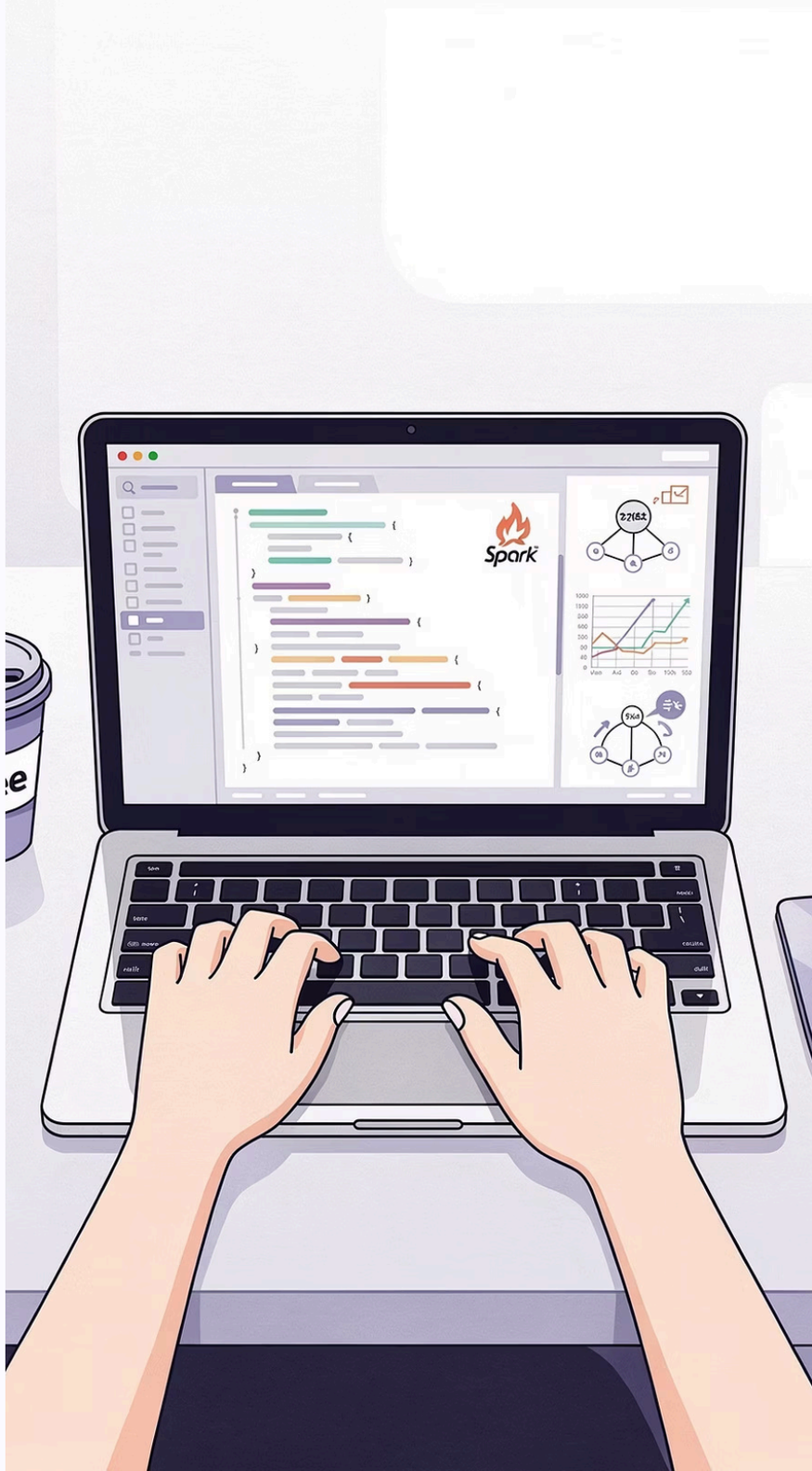
Set up a Kafka topic as the event source

02

Define a Structured Streaming read from Kafka in Spark

03

Apply transformations, aggregations, and windowing logic

04

Write results to Delta Lake with exactly-once guarantees